



Konzeption und Realisierung einer Android-basierten
Applikation für die Mitteilungen an der Hochschule Landshut

Yannick Weber, Matrikelnummer: 1043572

6. Dezember 2018

Inhaltsverzeichnis

1	Einführung	2
1.1	Beschreibung und Vision	2
1.2	Zielgruppe	2
2	Planung und Konzeption	3
2.1	Funktionale Anforderungen	3
2.1.1	Kursraumkonzept	3
2.1.2	Rollen	4
2.2	Anwendungsfälle für Studenten	5
2.2.1	Kursraum abonnieren	5
2.2.2	Kursraum de-abonnieren	5
2.2.3	Nachrichten ansehen	5
2.3	Anwendungsfälle für Dozenten	5
2.3.1	Meine Kursräume anzeigen	5
2.3.2	Kursraumübersicht anzeigen	5
2.3.3	Kursraum erstellen	5
2.3.4	Kursraum löschen	5
2.3.5	Nachricht in Kursraum einstellen	6
2.3.6	Nachricht aus Kursraum entfernen	6
2.4	Weitere Anforderungen	6
2.4.1	Filtern	6
2.4.2	Aktualisieren	6
2.4.3	Offline Verfügbarkeit	6
2.4.4	Caching	6
3	Oberflächen	7
3.1	Authentifizierung	7
3.2	Oberfläche für Dozenten	8
3.3	Oberfläche für Studenten	10
4	Backend	12
4.1	Datenbank	12
4.2	REST-API	13
5	Release-Planung	15
6	Tests	16
7	Wirtschaftlichkeit der App	16

Kapitel 1

Einführung

1.1 Beschreibung und Vision

An der Hochschule Landshut studieren gut fünftausend Studenten. Darüber hinaus umfasst das Personal annähernd 120 Professoren und Professorinnen und circa 45 Wissenschaftliche Mitarbeiter. Die Organisation eines reibungslosen Ablaufes des täglichen Studienbetriebes stellt also eine herausfordernde Aufgabe dar. Allerdings obliegt auch eine perfekt durchdachte Planung gewissen Ungenauigkeiten, die sich beispielsweise auf höhere Gewalt oder Krankheit zurückführen lassen. Es ist also nötig kurzfristige Änderungen an alle Teilnehmer möglichst schnell zu übermitteln. Aus diesem Grund ist eine verlässliche und komfortable Möglichkeit zur Kommunikation unerlässlich, um diese organisatorischen Herausforderungen bewältigen zu können.

Die Applikation “Notice Board” soll ein Werkzeug für eben jene Kommunikation darstellen. Speziell die 1:n Informationsübermittlung – ein Professor will beispielsweise einem komplettem Kurs eine Raumänderung mitteilen – soll im Fokus stehen. Aktuell wird dieser Anwendungsfall über das “schwarze Brett” abgewickelt; jedem Student eines ganzen Studiengangs werden alle Nachrichten mitgeteilt, auch wenn diese für bestimmte Studenten nicht relevant sind.

Zur Verbesserung der jetzigen Situation verfolgt “Notice Board” den Ansatz der sogenannten Kursräume. Ein dazu privilegierter Benutzer A erstellt eine Themengruppe und andere Anwender können diese abonnieren; sie erhalten also alle Nachrichten, die Benutzer A in diese Themengruppe einstellt. Im regulärem Fall übernehmen Professoren die Rolle des Kursräume; idealerweise wird für jeden Kurs eine Themengruppe erstellt. Die Studenten abonnieren dann die Kursräume der Kurse, die sie besuchen.

Um missbräuchliche Benutzung zu vermeiden werden bestimmte Rollen implementiert:

- **Dozent:** Darf Kursräume erstellen und Nachrichten einstellen.
- **Student:** Darf Kursräume abonnieren und deabonnieren.

1.2 Zielgruppe

Aktuell beschränkt sich die Zielgruppe auf Nutzer, die ein internetfähiges Mobiltelefon mit einer aktuellen Version des Betriebssystems Android besitzen. Bei der Zielgruppe dieser Anwendung sind drei Gruppen von Nutzern zu betrachten. Die erste und größte Nutzergruppe verwendet die App im Studienalltag um aktuelle organisatorische Informationen zu erhalten. Die zweite Nutzergruppe setzt die App zum verbreiten von Informationen an die erste Nutzergruppe ein. Die dritte und kleinste Nutzergruppe verwaltet lediglich die Nutzerrollen innerhalb des System; sie ist also bei der Zielgruppenanalyse zu vernachlässigen.

Zur ersten Nutzergruppe zählen größtenteils immatrikulierte Studenten der Hochschule Landshut, jedoch können auch Gasthörer oder Austauschstudenten die App benutzen. Zur zweiten Nutzergruppe zählen Dozenten, Lehrbeauftragte und Gastdozenten ferner auch Tutoren der Hochschule Landshut, ihnen wird mit Hilfe der App die Kommunikation mit ihren Studenten erleichtert.

Kapitel 2

Planung und Konzeption

2.1 Funktionale Anforderungen

Die Kursräume stellen das Kernkonzept dar; sie können erstellt, gelöscht, abonniert, deabonniert und Nachrichten in sie eingestellt werden. Ein Benutzer muss sich beim Start der Applikation mit seinem eindeutigem Nutzernamen und dem zugehörigem Passwort authentifizieren. Die Anwender sind in drei Kategorien eingeteilt: Student, Professor und Administrator. Je nach Rolle des Benutzers wird die jeweilige Oberfläche angezeigt. Im Folgenden wird das Kursraumkonzept und die jeweiligen Rollen genau beschrieben.

2.1.1 Kursraumkonzept

Ein Kursraum ist im Normalfall einem Kurs, der aus einem Professor und einer beliebigen Anzahl an Studenten besteht, zugeordnet. Er wird vom Dozenten erstellt und die Studenten können ihn anschließend abonnieren. Stellt der Kursraumersteller nun eine Nachricht ein, wird diese an jeden Abonnenten weitergeleitet. Ein Dozent kann eine beliebige Anzahl an Kursräumen erstellen und von ihm erstellte auch wieder löschen. Ein Student kann mehrere Kursräume abonnieren und wieder deabonnieren. Abbildung 2.1 visualisiert das eben Erläuterte.

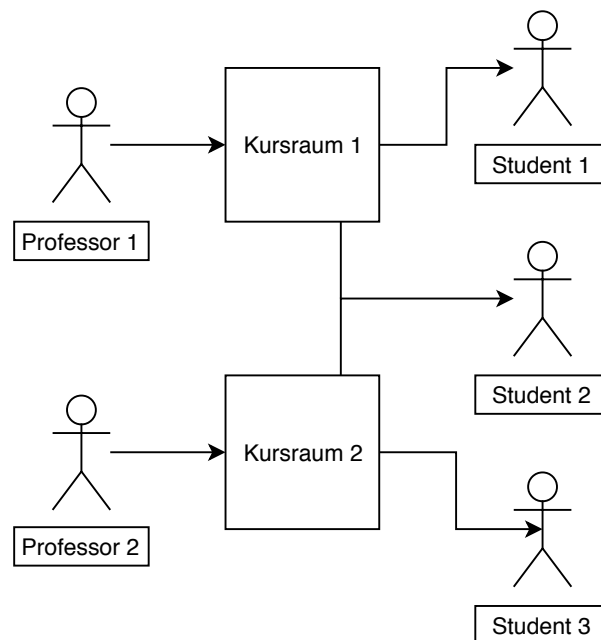


Abbildung 2.1: Visualisierung des Kursraumkonzeptes

2.1.2 Rollen

In den folgenden Abschnitten werden die Benutzerrollen und die speziellen Anwendungsfälle genau definiert. Alle Benutzer haben gemein, dass sie sich vor dem Benutzen der Applikation mit Passwort und Benutzernamen authentifizieren müssen. Die jeweiligen Rollen sind in der Datenbank hinterlegt.

Dozent

Ein Dozent kann Kursräume erstellen und diese wieder löschen. In einem von ihm erstelltem Raum kann er Nachrichten einstellen, die an alle Abonnenten dieses Raumes weitergeleitet werden.

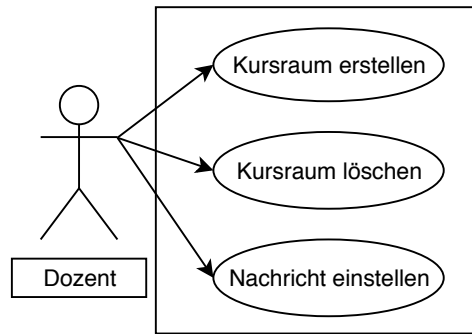


Abbildung 2.2: Berechtigungen für Benutzer in der Rolle "Dozent"

Student

Ein Student ist dazu berechtigt Kursräume zu abonnieren und zu deabonnieren. Er empfängt alle Nachrichten, die in von ihm abonnierten Kursräumen eingestellt wurden. Es können nur Kursräume deabonniert werden, die zuvor abonniert wurden.

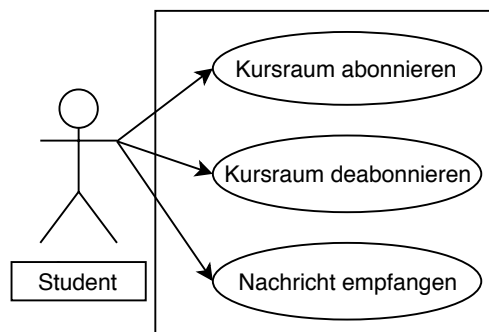


Abbildung 2.3: Berechtigungen für Benutzer in der Rolle "Student"

2.2 Anwendungsfälle für Studenten

2.2.1 Kursraum abonnieren

Use-Case	Ich möchte über eine einfache Gesamtansicht alle verfügbaren Kursräume betrachten und abonnieren können, sodass diese meinem Account hinzugefügt werden.
MuSCoW	MUST

Tabelle 2.1: Anwendungsfall Abonnieren

2.2.2 Kursraum de-abonnieren

Use-Case	Ich möchte in einer einfachen Gesamtansicht alle Kursräume betrachten, die ich abonniert habe und sie gegebenenfalls de-abonnieren.
MuSCoW	MUST

Tabelle 2.2: Anwendungsfall De-Abonnieren

2.2.3 Nachrichten ansehen

Use-Case	Ich möchte alle Nachrichten aller Kursräume, die ich abonniert habe betrachten. Diese sollen nach Einstellzeitpunkt sortiert sein.
MuSCoW	MUST

Tabelle 2.3: Anwendungsfall Nachrichten ansehen

2.3 Anwendungsfälle für Dozenten

2.3.1 Meine Kursräume anzeigen

Use-Case	Ich möchte in einer Übersicht alle Kursräume sehen, die ich erstellt habe.
MuSCoW	MUST

Tabelle 2.4: Anwendungsfall erstellte Kursräume anzeigen

2.3.2 Kursraumübersicht anzeigen

Use-Case	Ich in einer Kursraumansicht alle Nachrichten sehen, die ich in diesen Raum eingestellt habe.
MuSCoW	MUST

Tabelle 2.5: Anwendungsfall Kursraumübersicht anzeigen

2.3.3 Kursraum erstellen

Use-Case	Ich möchte einen Kursraum erstellen, den meine Studenten abonnieren können.
MuSCoW	MUST

Tabelle 2.6: Anwendungsfall Kursraum erstellen

2.3.4 Kursraum löschen

Use-Case	Ich möchte einen erstellten Kursraum löschen.
MuSCoW	MUST

Tabelle 2.7: Anwendungsfall Kursraum löschen

2.3.5 Nachricht in Kursraum einstellen

Use-Case	Ich möchte eine neue Nachricht in einen Kursraum, den ich erstellt habe, einstellen.
MuSCoW	MUST

Tabelle 2.8: Anwendungsfall Nachricht in Kursraum einstellen

2.3.6 Nachricht aus Kursraum entfernen

Use-Case	Ich möchte eine Nachricht aus einen Kursraum, den ich erstellt habe, entfernen.
MuSCoW	MUST

Tabelle 2.9: Anwendungsfall Nachricht aus Kursraum löschen

2.4 Weitere Anforderungen

2.4.1 Filtern

Requirement	In einer Übersichtsansicht (Kursräume, Nachrichten etc.) sollen die Dargestellten Daten gefiltert werden können.
MuSCoW	SHOULD

Tabelle 2.10: Anforderung Filtern

2.4.2 Aktualisieren

Requirement	In einer Übersichtsansicht (Kursräume, Nachrichten etc.) sollen die dargestellten Daten manuell aktualisiert werden könne.
MuSCoW	SHOULD

Tabelle 2.11: Anforderung Aktualisierung

2.4.3 Offline Verfügbarkeit

Requirement	Verfügt das Gerät über keine Internetverbindung sollen die zuletzt abgefragten Daten angezeigt werden. Darüberhinaus ist ein Hinweis einzublenden, der den Anwender darauf hinweist, das die Daten eventuell nicht aktuell sind.
MuSCoW	COULD

Tabelle 2.12: Anforderung Aktualisierung

2.4.4 Caching

Requirement	Die Daten sollen für einen gewissen Zeitraum – ungefähr 5 Minuten – zwischengespeichert werden, damit keine unnötigen Abfragen an das Backend gestellt werden müssen.
MuSCoW	COULD

Tabelle 2.13: Anforderung Aktualisierung

Kapitel 3

Oberflächen

Im Nachfolgendem werden die Benutzeroberflächen der Applikation, die enthaltenen Bedienelemente und deren jeweilige Funktion beschrieben.

3.1 Authentifizierung

Zu Beginn einer Sitzung muss sich jeder Anwender – unabhängig von seiner Rolle – mit seiner E-Mailadresse und dem zugehörigem Passwort anmelden. Es sind nur E-Mailadressen der Hochschule Landshut zulässig. Die Bedienungsfläche für den ersten Benutzungsschritt wird in Abbildung 3.1 dargestellt. Im Feld “Email” wird die Mailadresse des Anwenders und analog dazu im Feld “Passwort” das Passwort eingetragen. Durch Klick auf den Knopf “Sign In” startet ein Authentifizierungsversuch. Zunächst überprüft die Applikation die syntaktische Korrektheit der Mailadresse und informiert gegebenenfalls über ungültige Eingaben. Im Falle einer validen Eingabe werden die Anmeldedaten an den Server geleitet und mit der Datenbank abgeglichen. Existiert ein Benutzer mit den jeweiligen Daten, so wird die zugeordnete Rolle übermittelt. Abhängig von der Antwort des Servers öffnet sich dann die Oberfläche “Oberfläche für Dozenten” oder “Oberfläche für Studenten”.

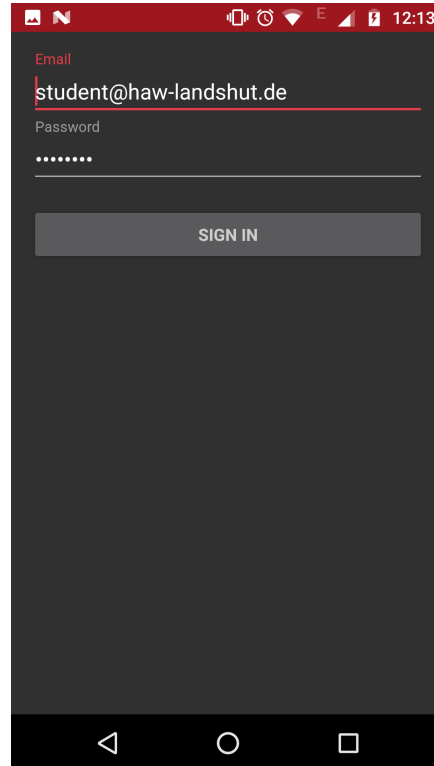


Abbildung 3.1: Oberfläche für das Anmelden in der Applikation

3.2 Oberfläche für Dozenten

Hat sich ein Nutzer der Rolle “Dozent” erfolgreich angemeldet, so erscheint die Oberfläche “My-Classrooms”. Hier werden alle Klassenräume angezeigt, die er verwaltet. Der Dozent hat nun die Möglichkeit entweder über den Button mit dem Löschen-Symbol Klassenräume und alle enthaltenen Nachrichten zu löschen (siehe Abbildung 3.2) oder über den Button unten Rechts einen neuen Klassenraum zu erstellen (siehe Abbildung 3.3).

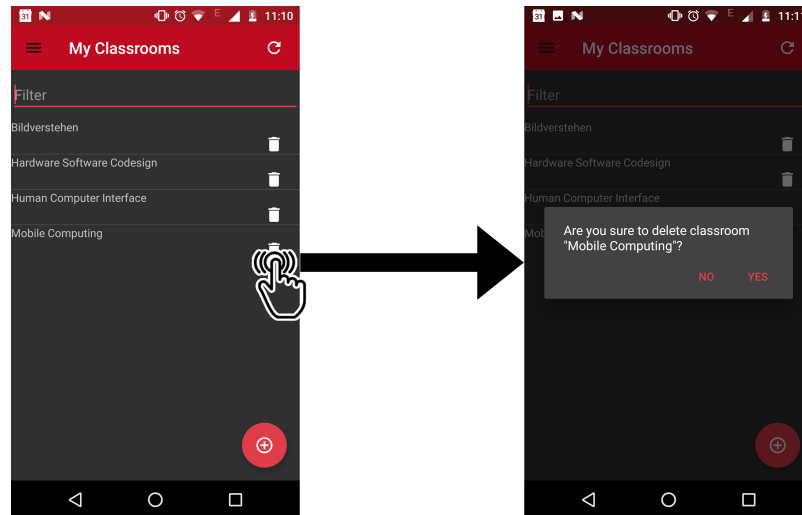


Abbildung 3.2: Löschen eines Kursraums für Benutzer in der Rolle “Dozent”

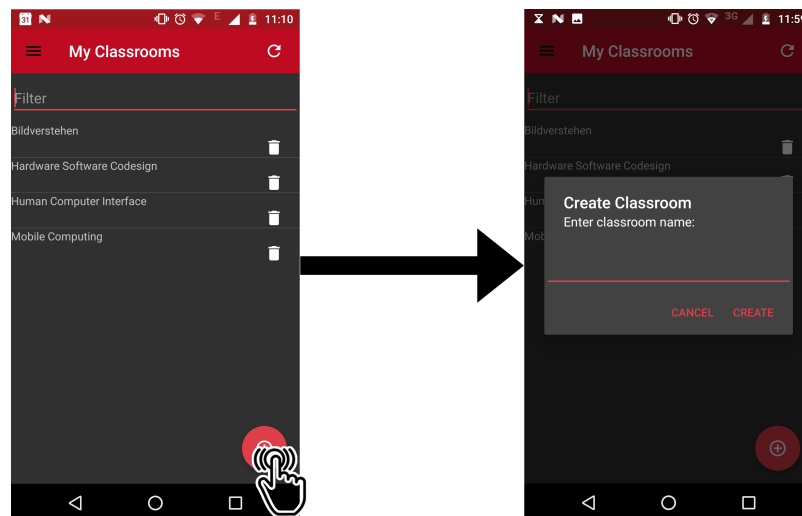


Abbildung 3.3: Erstellen eines Kursraums in der Rolle “Dozent”

Durch Klicken auf den Namen eines Kursraums wird in die Übersichtansicht für den gewählten Kursraum (siehe Abbildung 3.4). Es werden nun alle Nachrichten angezeigt, die der Dozent in der Vergangenheit in den Kursraum eingestellt hat. Nun besteht die Möglichkeit eine neue Nachricht zu erstellen. Hierfür muss der “erstellen”-Button in der Ansicht unten rechts gedrückt werden. Nun erscheint ein Dialog, in dem der Inhalt der neuen Nachricht eingegeben werden kann. Mit “Okay” wird das Erstellen der Nachricht bestätigt; sie ist nun für alle Studenten, die diesen Kursraum abonniert haben sichtbar. Für genaueres Verständnis ist der Ablauf dieser Benutzerinteraktion in Abbildung 3.5 dargestellt. Möchte der Dozent eine Nachricht aus dem Kursraum entfernen, so betätigt er den “Löschen” Button neben der betreffenden Nachricht. Nun erscheint ein Dialog, der den Benutzer zu erneutem Bestätigten der Löschung aufruft. Bestätigt der Anwender mit der Option “Delete” so wird die Nachricht gelöscht.

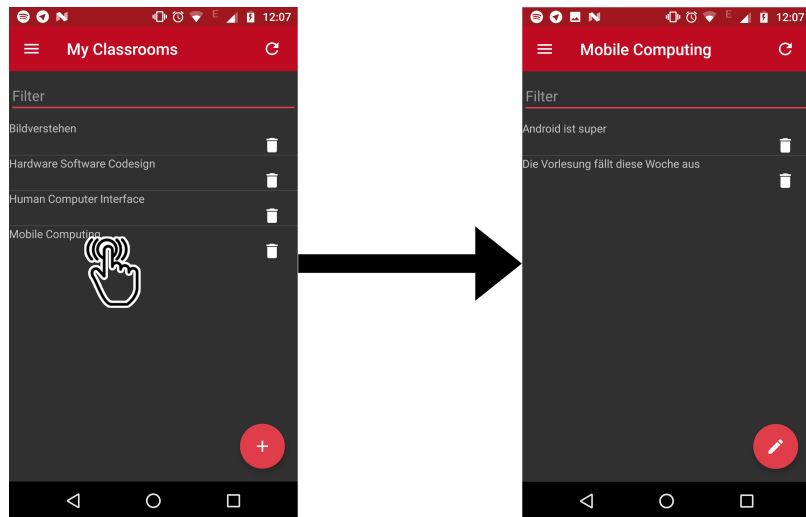


Abbildung 3.4: Wechseln zur Übersicht über einen Kursraum für Benutzer in der Rolle “Dozent”

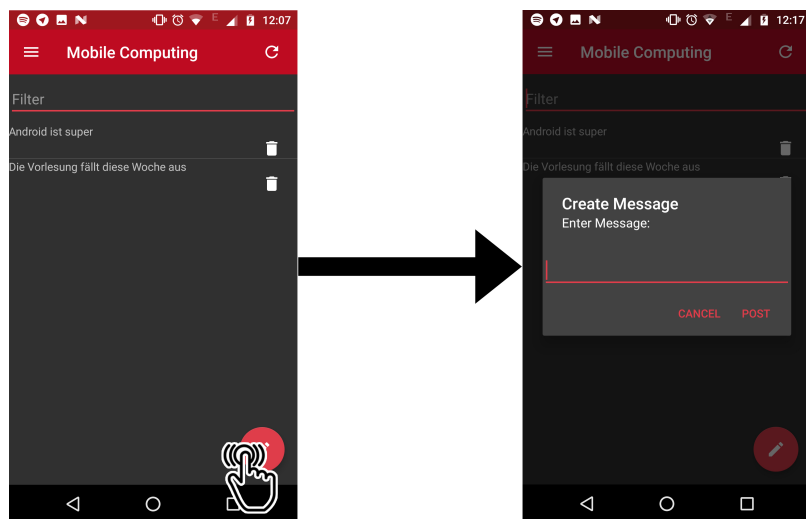


Abbildung 3.5: Einstellen einer Nachricht in einem Kursraum für Benutzer in der Rolle “Dozent”

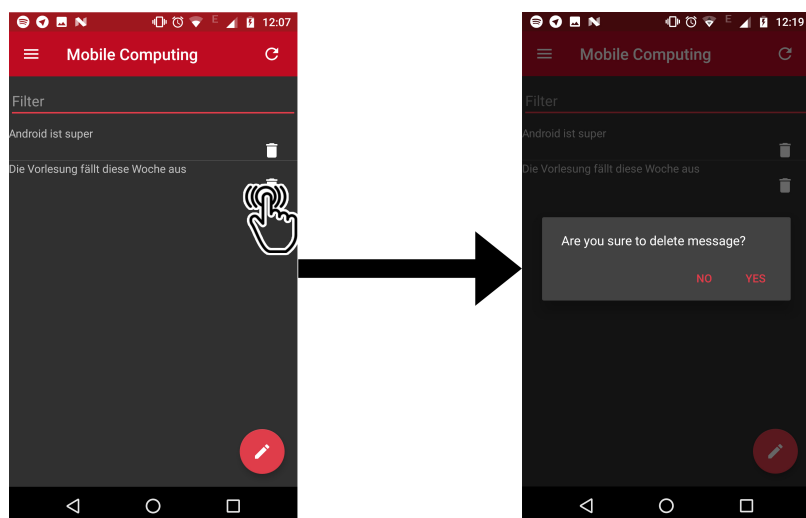


Abbildung 3.6: Löschen einer Nachricht in einem Kursraum für Benutzer in der Rolle “Dozent”

3.3 Oberfläche für Studenten

Nach erfolgreicher Anmeldung wird einem Studenten die Oberfläche aus Abbildung 3.7 gezeigt. Hier werden alle Nachrichten aus seinen abonnierten Kursräumen – sortiert nach Einstellzeitpunkt – angezeigt. Die angezeigten Daten werden sofort nach betreten dieser Ansicht vom Server geladen. Durch Drücken auf das “Refresh” Symbol oben rechts, werden die Nachrichten erneut vom Server abgerufen und eventuell zwischenzeitlich neu eingestellte ebenfalls angezeigt. Der Menü-Button oben links führt den Benutzer in die Ansicht aus Abbildung 3.8. Zur Verbesserung der Übersichtlichkeit kann in dem Texteingabefeld “Filter” ein Suchtext eingegeben werden; es werden so nur noch Nachrichten angezeigt, deren Kursraum oder Nachrichtentext die eingegebene Zeichenfolge enthält.

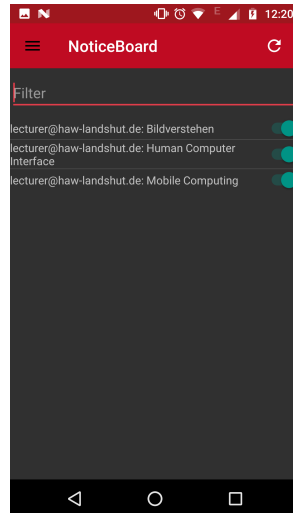


Abbildung 3.7: Hauptmenü für Benutzer in der Rolle “Student”

Studenten können mittels des in Abbildung 3.8 gezeigten Screens durch die App navigieren. Es können mit “My Messages” alle aktuellen Nachrichten der abonnierten Kursräume angezeigt werden. Unter “My Classrooms” werden alle abonnierten Klassenräume geladen, diese können hier deabonniert werden. Über die Option “All Classrooms” werden alle offenen Kursräume angezeigt; ein Student hat so die Möglichkeit diese zu abonnieren beziehungsweise zu deabonnieren. Über den Menüpunkt “logout” meldet sich der Anwender aus der Applikation ab und gelangt zurück in die Authentifizierungsansicht.

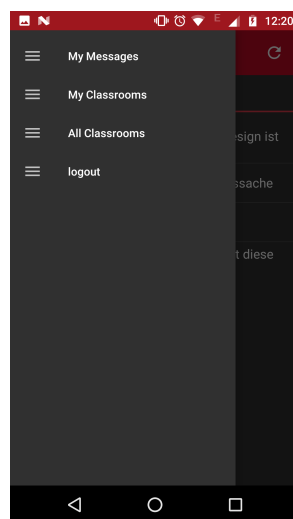


Abbildung 3.8: Hauptmenü für Benutzer in der Rolle “Student”

In der Ansicht, die in Abbildung 3.9 dargestellt ist, werden Benutzern in der Rolle “Student” alle Kursräume und der zugeordnete Dozent angezeigt, die sie abonniert haben. Rechts neben jedem abonnierten Kursraum befindet sich ein Schalter, der anzeigt, ob der betreffende Klassenraum abonniert ist oder nicht. Bei Betreten der Ansicht werden alle Kursräume als abonniert markiert. Durch Betätigen des Schalters wird der entsprechende Kursraum deabonniert und der Schalter steht auf aus. Es besteht die Möglichkeit durch erneutes Drücken den Kursraum wieder zu abonnieren; erst bei manueller Aktualisierung mittels des “Refresh”-Symbols wird der Kursraum entfernt. Filtern analog zur Oben beschriebenen Oberfläche ist ebenfalls möglich.

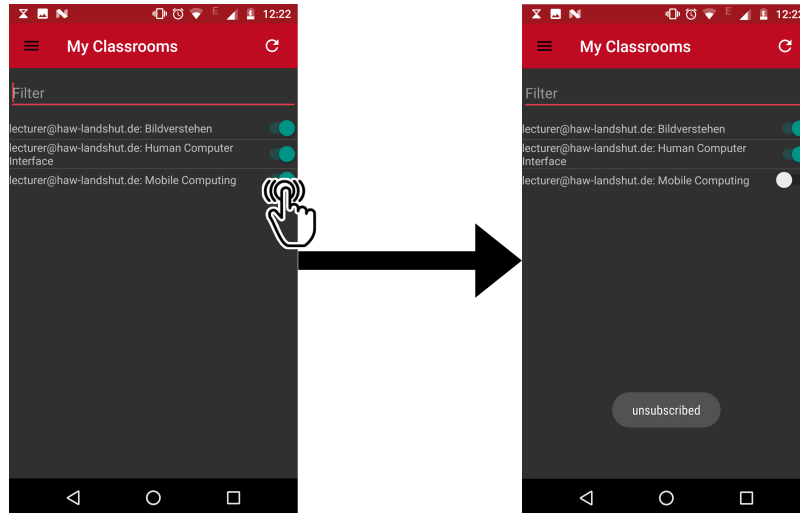


Abbildung 3.9: Deabonnieren eines Kursraums für Benutzer in der Rolle “Student”

Die Activity “All Classrooms” (Abbildung 3.10) zeigt alle Kursräume, die vom Benutzer abonniert werden können. Nicht abonnierte Kursräume werden mit einem deaktiviertem Schalter markiert und umgekehrt. Der Anwender hat hier die Möglichkeit weitere Kursräume zu suchen und abonnieren oder gegebenenfalls auch zu de-abonnieren. Bei Betreten dieser Activity werden die aktuellen Daten vom Server abgerufen, Betätigen des “Refresh”-Symbols aktualisiert die Daten manuell. Über das Menü Symbol gelangt der Benutzer zurück in die Navigation.

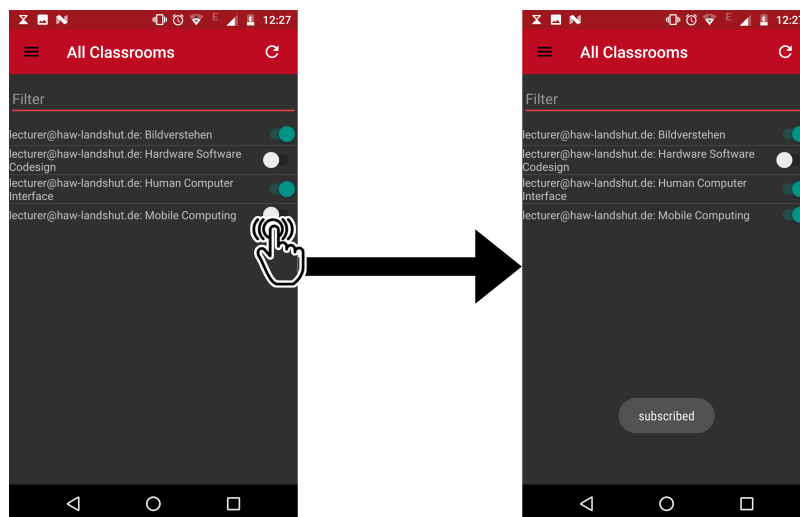


Abbildung 3.10: Abonnieren eines Kursraums für Benutzer in der Rolle “Student”

Kapitel 4

Backend

Damit die Nutzer der App interaktiv miteinander kommunizieren können, ist eine Verbindung zu einem Server (im weiteren als Backend bezeichnet) nötig. Daten wie Klassenräume und Nachrichten können unter Angabe von validen Anmeldedaten abgefragt werden. Als Schnittstelle zwischen der App und dem Backend findet eine REST-API Verwendung. Die Datenbank verwaltet alle Daten wie Klassenräume, eingestellte Nachrichten und Anmeldedaten. Eine Grundsätzliche Struktur der Architektur der Komposition von SQL-Datenbank, REST-API und App ist in Abbildung 4.1 dargestellt.

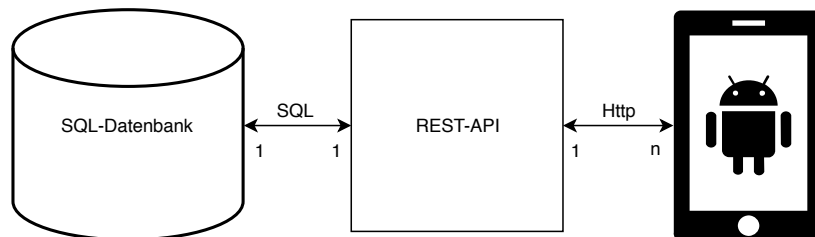


Abbildung 4.1: Architektur des Backends

4.1 Datenbank

Zur Verwaltung der Anmeldedaten, Klassenräume, Nachrichten und Abonnements wird eine relationale Datenbank eingesetzt. Die Tabelle “Persons” enthält die Anmeldedaten (E-Mailadresse und Passwort) jedes Nutzers, sowie dessen jeweilige Rolle (Student, Dozent, Admin). In “Messages” sind die Nachrichten gespeichert, die Dozenten in ihre Kursräume einstellen. Jede Nachricht ist einem Kursraum eindeutig zugeordnet und erhält eine eindeutige id. Im Feld “Payload” ist der eigentliche Text der Nachricht abgespeichert. Kursraum werden in der Tabelle “Classrooms” verwaltet. Jedem Raum ist ein Dozent eindeutig zugeordnet. Über die “subscribes” wird die n:m-Beziehung zwischen Studenten und Kursräumen modelliert; sie enthält die Abonnements aller Studenten. Abbildung 4.2 zeigt detailliert die Struktur der Datenbank.

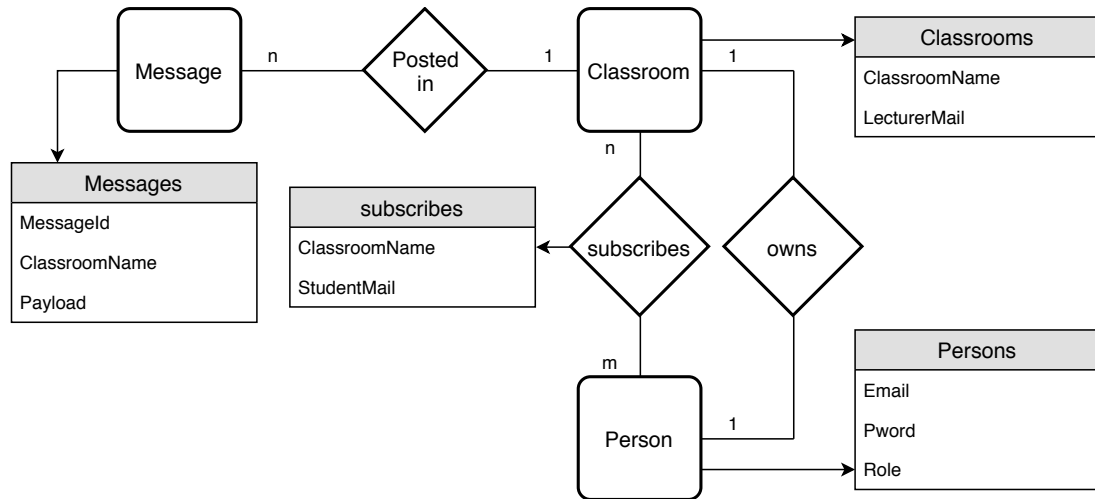


Abbildung 4.2: Entity-Relationship Modell der Datenbank

4.2 REST-API

Als Schnittstelle zwischen Backend und App dient eine RESTful-API. Verschiedene Daten sind über Endpoints (siehe Tabelle 4.1) abrufbar, welche jeweils mit einer eindeutigen URI adressiert sind. Jeder dieser Endpoints ist einem bestimmtem Anwendungsfall zugeordnet, zum Abrufen der Daten muss der Http-Client einen POST-Request senden, der die geforderten Argumente im JSON-Format enthält. Für Besseres Verständnis ist in Abbildung 4.3 ein Beispiel angegeben: für den Login sendet die App einen POST-Request, welcher die Anmeldedaten enthält; der Server antwortet dann mit der Rolle des zugehörigen Benutzers. Der Vorteil dieser Art von Schnittstelle liegt darin, dass das Backend *stateless* – also ohne Wissen über den Status des Clients – implementiert werden kann. Jedoch müssen bei der Abfrage jeder Ressource die Anmeldedaten des Benutzers gesendet werden. Darüber hinaus gestaltet sich so die Umsetzung der Serverstruktur als einfach, da sie so ohne Nebeneffekt erweiterbar ist. Der Hauptgrund für die Wahl einer RESTful-Schnittstelle ist jedoch, dass sie unabhängig vom System implementiert werden kann. Es wäre also beispielsweise ohne weiteres möglich neben der App auch eine Webanwendung zu erstellen, welche sich ebenfalls mit dem Backend verbindet.

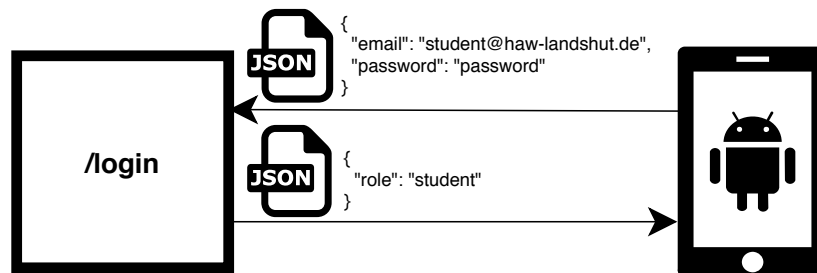


Abbildung 4.3: Beispiel für Kommunikation mit der REST-API

Endpoint URI	Argumente	Rückgabewert
/classrooms/all	Student-Anmeldedaten	alle Kursräume und Status
/classrooms/subscribed	Student-Anmeldedaten	alle abonnierten Kursräume
/classrooms/subscribe	Student-Anmeldedaten, Kursraumname	Erfolg
/classrooms/unsubscribe	Student-Anmeldedaten, Kursraumname	Erfolg
/classrooms/create	Dozent-Anmeldedaten, Kursraumname	Erfolg
/classrooms/delete	Dozent-Anmeldedaten, Kursraumname	Erfolg
/classrooms/owned	Dozent-Anmeldedaten	alle Dozenten-Kursräume
/login	Anmeldedaten	Benutzerrolle
/messages/student	Student-Anmeldedaten	abonnierte Nachrichten
/messages/create	Dozent-Anmeldedaten, Kursraumname, Text	Erfolg
/messages/delete	Dozent-Anmeldedaten, id	Erfolg
/messages/classroom	Anmeldedaten, Kursraumname	Kursraumnachrichten

Tabelle 4.1: Liste aller Endpoints und deren Argumente beziehungsweise Rückgabewerte

Kapitel 5

Release-Planung

Im Groben sollen die einzelnen Entwicklungsschritte folgendermaßen geliedert werden:

- Vorbereitung der Werkzeuge (Android-Studio, IntelliJ) und erstellen von leeren aber lauffähigen Projekten
- Implementierung der Benutzerverwaltung und des Logins
- Implementierung der Funktionalitäten für Benutzer in der Rolle “Student”
- Implementierung der Funktionalitäten für Benutzer in der Rolle “Dozent”
- Kleinere Überarbeitung der Gestaltung der Benutzeroberflächen
- Implementierung von COULD-Anforderungen

Im Bezug auf den Arbeitsfluss ist geplant das Backend und die App Hand in Hand umzusetzen. Für jede Anforderung wird zuerst der Datenbankzugriff und der zugehörige REST-Endpoint implementiert. Im Anschluss wird dann die jeweilige Benutzeroberfläche, die sich auf die umgesetzte Anforderung bezieht, erstellt. In Tabelle 5.1 ist ein detaillierter Ablauf der Releases dargestellt.

Version	Inhalt
0.0	Projekte erstellen und IDEs, Git-Repositories aufsetzen
0.1.1	Login: Backend Datenbankzugriff implementieren
0.1.2	Login: Backend REST-Endpoints implementieren
0.1.3	Login: App Oberfläche implementieren
0.1.4	Login: App Datenabfrage vom Server implementieren
0.2.1	Student: Backend Datenbankzugriff implementieren
0.2.2	Student: Backend REST-Endpoints implementieren
0.2.3	Student: App Oberfläche implementieren
0.2.4	Student: App Datenabfrage vom Server implementieren
0.3.1	Dozent: Backend Datenbankzugriff implementieren
0.3.2	Dozent: Backend REST-Endpoints implementieren
0.3.3	Dozent: App Oberfläche implementieren
0.3.4	Dozent: App Datenabfrage vom Server implementieren
0.4	Oberflächen überarbeiten
0.5	Caching und Offline-Speicherung implementieren
1.0	finales Release

Tabelle 5.1: Geplanter Versionsverlauf der Releases

Kapitel 6

Tests

Die Tests des Gesamtsystems teilen sich in folgende Gruppen auf:

- **Datenbankzugriff:** Unter Anwendung des Testframeworks “JUnit” werden die Datenzugriffe – also die SQL-Abfragen – auf ihre Korrektheit geprüft. Darüberhinaus wird auch das Verhalten im Fehlerfall – wie ungültige Anmeldedaten – simuliert.
- **REST-API:** Mittels des in IntelliJ integrierten Http-Clients können definierte Http-Requests an die jeweiligen Endpoints gesendet werden; hiermit wird die fehlerfreie Funktion der REST-API sichergestellt. Zum Einen wird das Verhalten mit korrekten Eingabedaten (JSON-Bodies) zum Anderen das Erkennen von ungültigen bzw. unvollständigen Daten getestet.
- **App:** Zur Überprüfung des Verhaltens der App (Korrektes Navigieren, richtiges Verhalten der Buttons und fehlerfreies Kommunizieren mit dem Backend) werden manuelle Tests durchgeführt. Hierbei wird das alltägliche Benutzen der App nachgestellt und eventuell auftretende Fehler analysiert und gegebenenfalls beseitigt. Für die Studenten- und die Dozentenansicht wird jeweils ein Testaccount angelegt und die Anwendungsfälle dieser Rollen werden mehrmals pro Entwicklungsschritt zu Testzwecken ausgeführt.

Kapitel 7

Wirtschaftlichkeit der App

Damit sich die App möglichst weit bei den Nutzern verbreitet wird, soll sie kostenlos über die einschlägigen Verbreitungsdienste wie der “Google Play Store” angeboten werden. Im Sinne der Benutzerfreundlichkeit wird auch auf das Anzeigen von Werbung gänzlich verzichtet. Als Einnahmequelle soll jedoch das Verkaufen der Lizenz für das Backend dienen. Möchte eine Institution diese Applikation einsetzen so wird ihr darüber hinaus die Inbetriebnahme und der Support der Serverstruktur in Rechnung gestellt.

Aus der Marktanalyse geht hervor, dass bereits viele Apps, die sich der Kommunikation von Nutzern untereinander annehmen existieren. Allerdings bietet keine dieser Apps ein zum Kursraumkonzept ein vergleichbares Abonnier-System. Außerdem fehlt die genaue Anpassung an Anforderungen, die der Alltag des Betriebs an einer Hochschule stellt. Ein weiterer Vorteil, den diese App gegenüber anderen Kommunikationsapps hat, ist die Möglichkeit Benutzerrollen zu zentral zu verwalten und so Missbrauch wie Spam zu verhindern. Es ist also anzunehmen, dass die App “Notice Board” erfolgreich und gewinnbringend zu vermarkten ist.

Abbildungsverzeichnis

2.1	Visualisierung des Kursraumkonzeptes	3
2.2	Berechtigungen für Benutzer in der Rolle “Dozent”	4
2.3	Berechtigungen für Benutzer in der Rolle “Student”	4
3.1	Oberfläche für das Anmelden in der Applikation	7
3.2	Löschen eines Kursraums für Benutzer in der Rolle “Dozent”	8
3.3	Erstellen eines Kursraums in der Rolle “Dozent”	8
3.4	Wechseln zur Übersicht über einen Kursraum für Benutzer in der Rolle “Dozent”	9
3.5	Einstellen einer Nachricht in einem Kursraum für Benutzer in der Rolle “Dozent”	9
3.6	Löschen einer Nachricht in einem Kursraum für Benutzer in der Rolle “Dozent”	9
3.7	Hauptmenü für Benutzer in der Rolle “Student”	10
3.8	Hauptmenü für Benutzer in der Rolle “Student”	10
3.9	Deabonnieren eines Kursraums für Benutzer in der Rolle “Student”	11
3.10	Abonnieren eines Kursraums für Benutzer in der Rolle “Student”	11
4.1	Architektur des Backends	12
4.2	Entity-Relationship Modell der Datenbank	13
4.3	Beispiel für Kommunikation mit der REST-API	13

Tabellenverzeichnis

2.1	Anwendungsfall Abonnieren	5
2.2	Anwendungsfall De-Abonnieren	5
2.3	Anwendungsfall Nachrichten ansehen	5
2.4	Anwendungsfall erstellte Kursräume anzeigen	5
2.5	Anwendungsfall Kursraumübersicht anzeigen	5
2.6	Anwendungsfall Kursraum erstellen	5
2.7	Anwendungsfall Kursraum löschen	5
2.8	Anwendungsfall Nachricht in Kursraum einstellen	6
2.9	Anwendungsfall Nachricht aus Kursraum löschen	6
2.10	Anforderung Filtern	6
2.11	Anforderung Aktualisierung	6
2.12	Anforderung Aktualisierung	6
2.13	Anforderung Aktualisierung	6
4.1	Liste aller Endpoints und deren Argumente beziehungsweise Rückgabewerte	14
5.1	Geplanter Versionsverlauf der Releases	15